



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE



Introdução ao Design e Projeto de Arquitetura de Software

Nesta aula, vamos explorar os conceitos, princípios e práticas que moldam as estruturas e a organização de sistemas de software modernos.



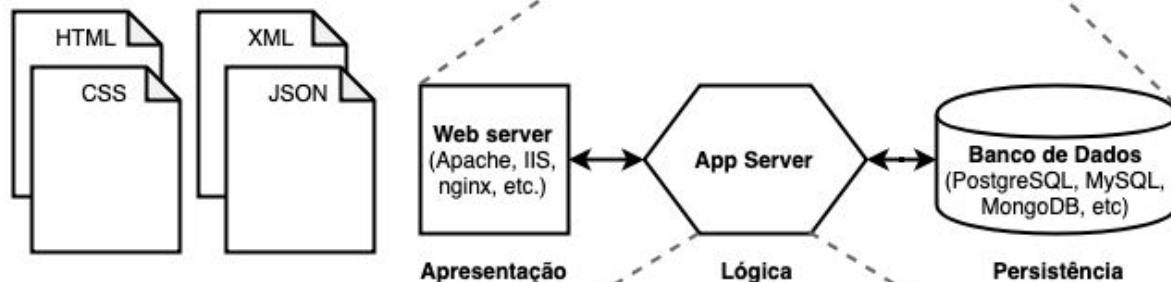
Cliente-Servidor vs P2P



HTTP & URIs

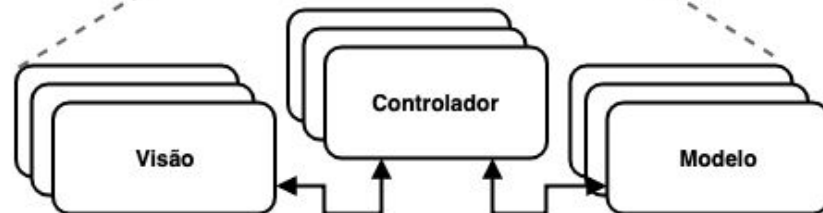
Arquitetura 3 camadas

Escalabilidade



Padrões Arquiteturais

Model-View- Controller



Padrões de Projeto

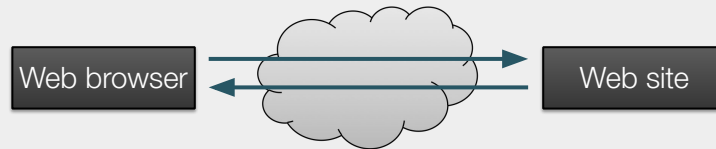
Idiomas



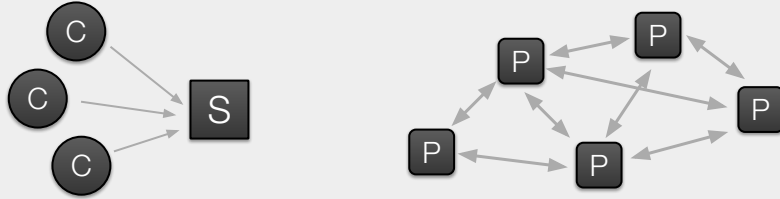
A Web

A Web é uma arquitetura cliente/servidor

É fundamentalmente uma estrutura orientada a requisição e resposta



Cliente-Servidor vs. Peer-to-Peer



- Cliente e Servidor são especializados em suas tarefas
 - Cliente: fazer perguntas em nome do usuário
 - Servidor: esperar e responder às perguntas, serve a muitos clientes
- Padrões de projeto (arquitetural, estrutural, computacional, etc) capturam soluções estruturais comuns para problemas recorrentes!
 - Cliente/Servidor é um padrão arquitetural!



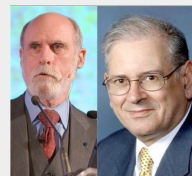
Protocolos TCP/IP

- Um endereço IP (Internet Protocol) identifica uma interface física de rede com quatro octeto, ex. **128.32.244.172**
- O endereço **127.0.0.1** é “este computador”, chamado de **localhost**, se não estiver conectado na internet!
- TCP/IP (Transmission Control Protocol/Internet Protocol) fornece abstração de um fluxo ordenado e confiável entre endereços IP
- TCP: torna o IP confiável através da detecção de pacotes “perdidos”, dados que chegam fora de ordem, erros de transmissão, redes lentas, etc, e respondem de forma apropriada
- Portas TCP permitem múltiplas apps no mesmo computador
- Vint Cerf & Bob Kahn: 2004 Turing Award for Internet architecture & protocols, incl. TCP/IP

GET /bears/

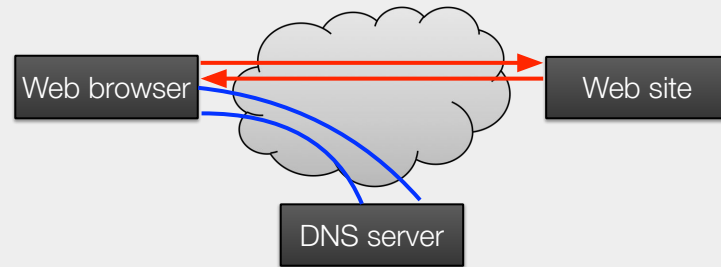


HTTP/0.9 200
OK



DNS

- A Web é uma arquitetura cliente/servidor
- É fundamentalmente uma estrutura orientada a requisição e resposta
- Domain Name System (DNS) é outro serviço que mapeia nomes para endereços IP



Hypertext Transfer Protocol

- Protocolo requisição/resposta baseado em ASCII para transmissão de informação na Web
- Requisição HTTP inclui
 - Método de requisição (GET, POST, etc)
 - Uniform Resource Identifier (URI)
 - Versão do protocolo HTTP entendido pelo cliente
 - Headers – informações extra sobre a requisição
- Resposta HTTP inclui
 - Protocol version & Status Code => => => => => => =>
 - Cabeçalho da resposta
 - Corpo da resposta
- HTTP é sem estado! portanto, cookies.

HTTP status codes:

2xx — *all is well*

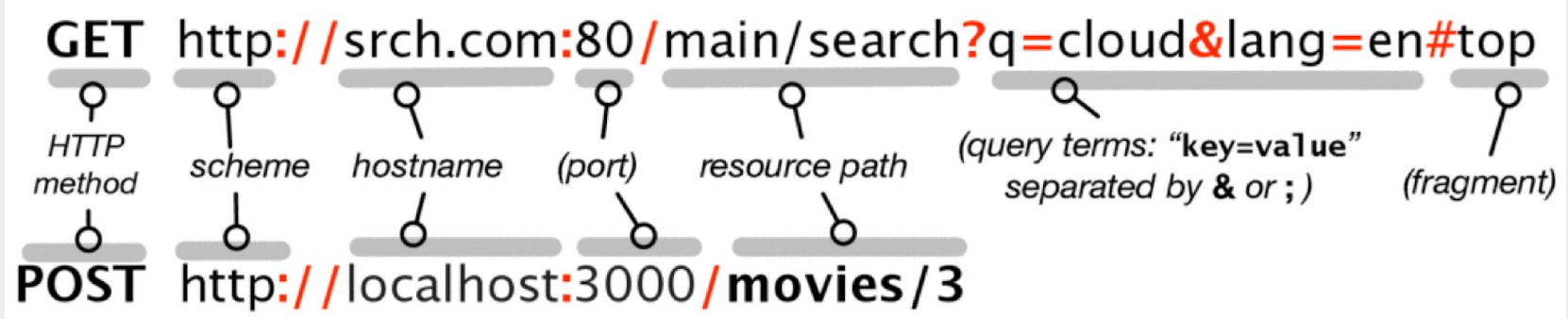
3xx — *resource moved*

4xx — *access problem*

5xx — *server error*



HyperText Transfer Protocol roda sobre TCP/IP



- HTTP method or verb:
 - GET, POST, PUT, PATCH, DELETE, HEAD
 - Idea: GETs should be **side-effect-free**
- Parameters considered separate from path
 - can also be carried in request body if a POST
- **Route = HTTP method + URI**



Cookies

- Problemas da Web 1.0: como guiar o usuário “através” de um fluxo de páginas?
- Usar endereço IP para identificar retorno de usuários?
 - Computadores públicos, usuário compartilham IP
- Embarcar identificação de usuários nas URI de busca?
 - Quebra o uso de cache
- Rapidamente substituído por cookies
 - Frameworks gerenciam as violações evidentes dos cookies para você



Sumário

- O TCP/IP fornece uma abstração de fluxo confiável entre 2 endpoints IP
- HTTP é um dos muitos protocolos de transporte cliente-servidor, requisição-resposta, que usa TCP/IP
- Uma solicitação HTTP (**cliente->servidor**) solicita um **recurso** especificando uma **rota** e possivelmente **cabeçalhos**
 - route = método HTTP + URI
- Uma resposta HTTP (**servidor->cliente**) fornece **status**, **cabeçalhos** e **corpo** (recurso)



Pergunta

Qual das afirmações a seguir é verdadeira sobre as duas requisições HTTP:

GET /foo/bar

POST /foo/bar

- A. Eles são indistinguíveis para um aplicativo SaaS
- B. Eles são distinguíveis e devem ter comportamentos diferentes
- C. Eles são distinguíveis e podem ter comportamentos diferentes
- D. Um determinado aplicativo pode manipular um ou outro, mas não ambos



Pergunta

Qual das afirmações a seguir é verdadeira sobre as duas requisições HTTP:

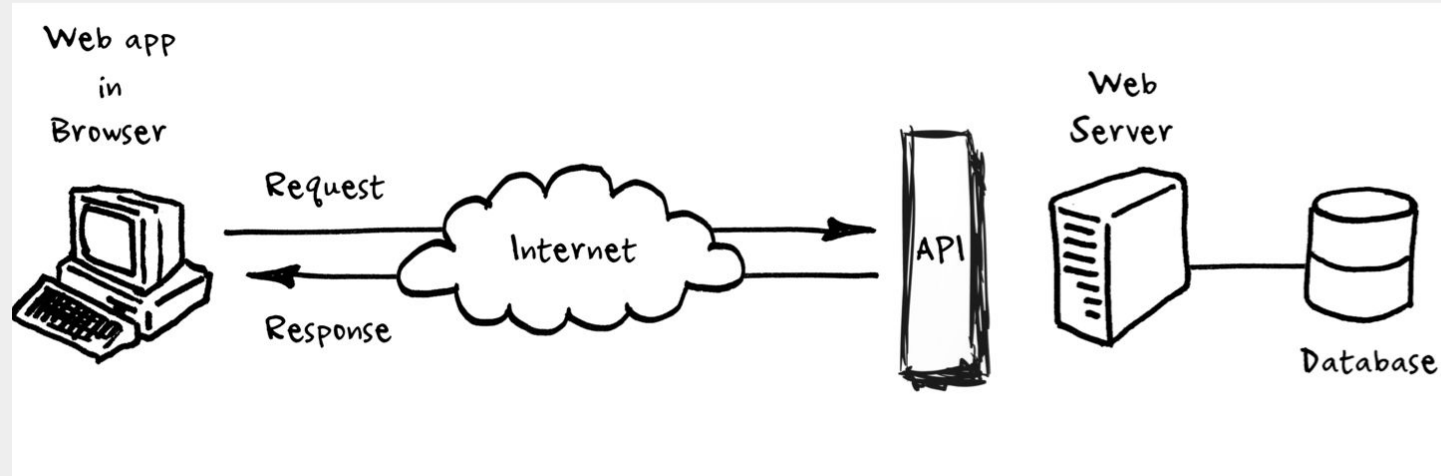
GET /foo/bar

POST /foo/bar

- A. Eles são indistinguíveis para um aplicativo SaaS
-  B. Eles são distinguíveis e devem ter comportamentos diferentes
- C. Eles são distinguíveis e podem ter comportamentos diferentes
- D. Um determinado aplicativo pode manipular um ou outro, mas não ambos



Arquitetura 3 Camadas



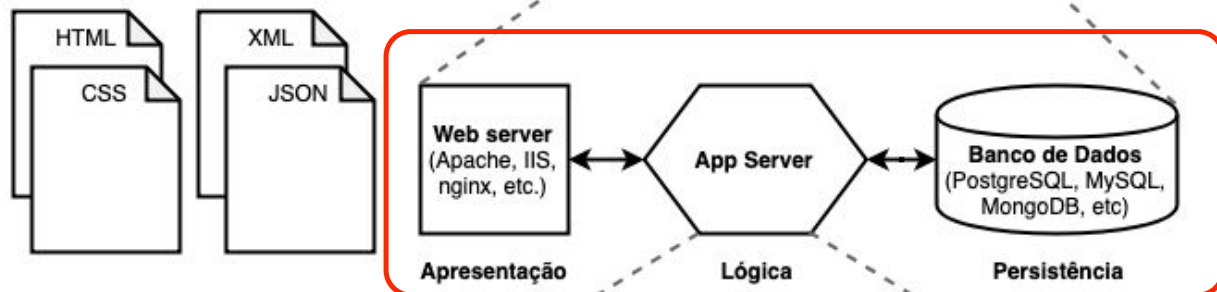
Cliente-Servidor vs P2P



HTTP & URIs

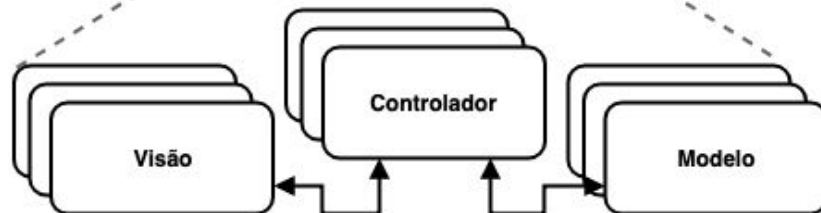
Arquitetura 3 camadas

Escalabilidade



Padrões Arquiteturais

Model-View- Controller

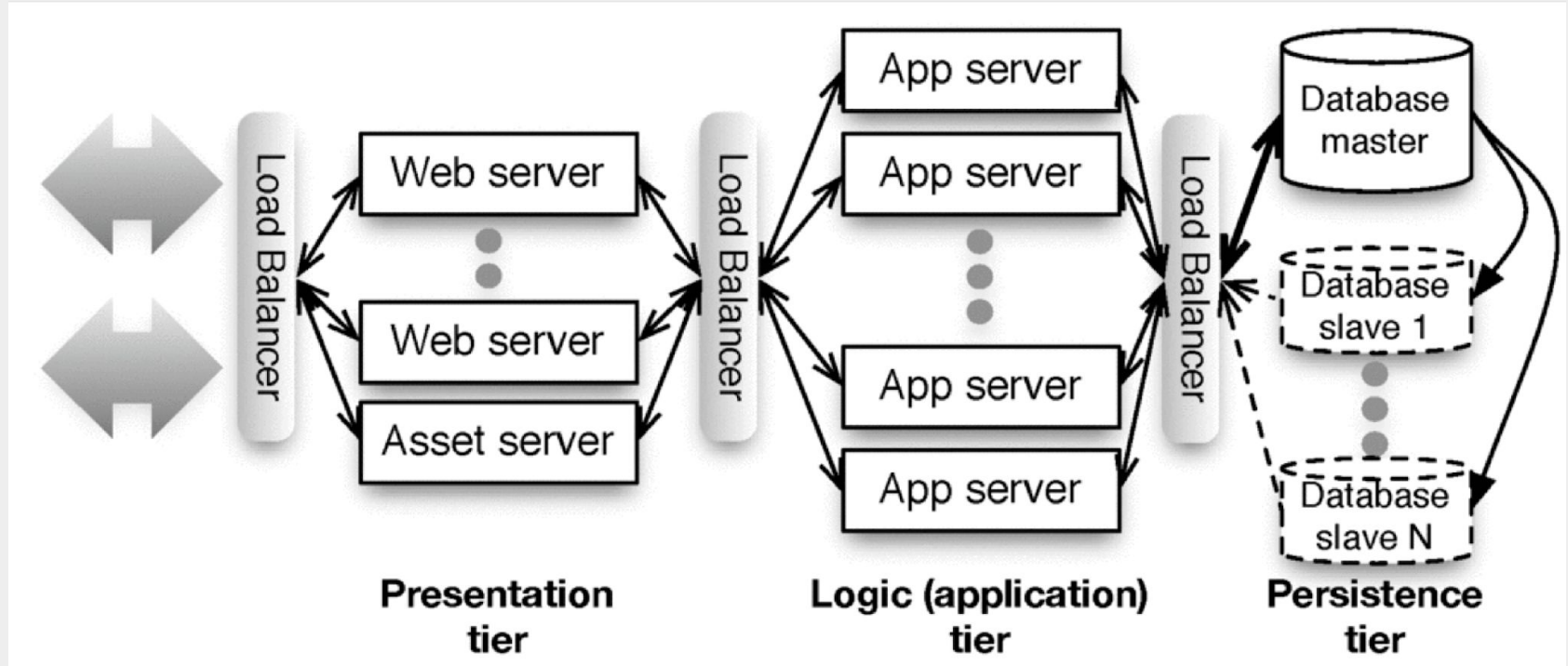


Padrões de Projeto

Idiomas



“Shared-nothing”



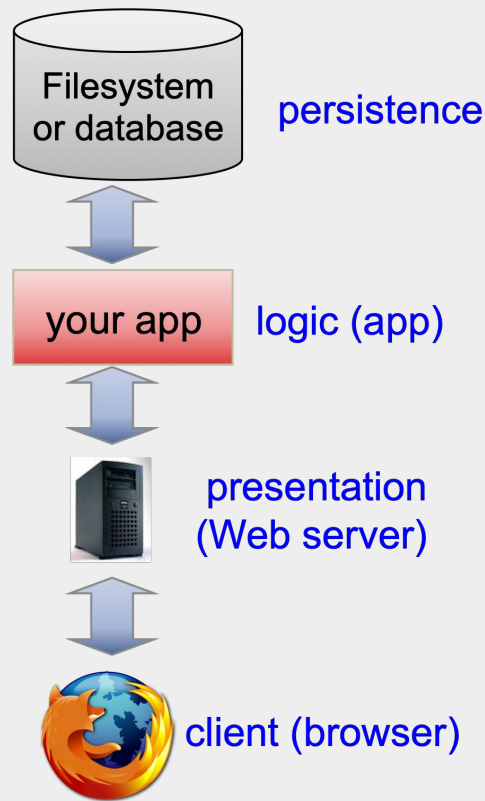
Geração de conteúdo dinâmico

- Antigamente a maioria das páginas da web eram (coleções de) arquivos antigos simples
- Mas os sites mais interessantes da Web1.0/e-commerce, na verdade, executavam um programa para gerar a “página”
- Originalmente: templates com "trechos" de código incorporados
- Eventualmente, o código tornou-se "rabo que abanou o cachorro" e mudou-se para fora do servidor Web



Sites que são programas (SaaS)

- Como você:
 - “mapeia” a URI para o programa e função correta?
 - Passa argumentos?
 - Evoca programas em um servidor?
 - Trata armazenamento persistente?
 - Trata cookies?
 - Trata erros?
 - Empacota saídas de volta para o usuário?
- Frameworks dão suporte a estas tarefas comuns



Sumário: Web 1.0 SaaS

- Browser requisita recursos (URI) usando HTTP
 - HTTP é um protocolo simples de requisição-resposta que depende do TCP/IP
 - Em SaaS, a maioria das URIs disparam eventos de uma aplicação
- HTML é utilizado para codificar conteúdo, CSS para estilizar o visual
- Cookies permitem ao servidor rastrear os clientes
 - O browser automaticamente envia o cookie para o servidor em cada requisição
 - O servidor pode atualizar o cookie em cada resposta
- Frameworks abstraem tudo isso de maneira conveniente para os desenvolvedores
- ...e ajuda a mapear SaaS para uma arquitetura 3-camadas, shared-nothing



AJAX chegou (Asynchronous JavaScript And XML, aka “Web 2.0”)

- Antes: a resposta HTTP contém conteúdo a ser exibido no navegador
- Depois: a resposta HTTP contém dados
 - O navegador organiza os dados de resposta para uma função de manipulador JavaScript específica
 - Essa função pode usar dados para modificar o que já está sendo exibido (“aplicativo de página única”)
- Então requisição/resposta HTTP <-> chamando uma função!
- Web como conjunto de serviços compostos independentes: Arquitetura orientada a serviços



Arquitetura de Software

Você pode/consegue **projetar** um software de forma que seja possível **recombinar** módulos **independentes** para ofertar **diferentes** apps sem muito **esforço** de programação?



Arquitetura Orientada a Serviço

- SOA: Arquitetura de Software onde todos os componentes são projetados como serviço
- Apps são compostas por serviços interoperáveis
 - Fácil de derivar uma nova versão para um conjunto de usuários
 - Também é fácil para se recuperar de um erro de projeto
- Contraste para “SW silo” sem uma API interna



CEO: Amazon shall use SOA!

1. “All teams will henceforth expose their data and functionality through service interfaces.
2. “Teams must communicate with each other through these interfaces.
3. “There will be no other form of interprocess communication allowed: no direct linking, no direct reads of another team's data store, no shared-memory model, no back-doors whatsoever. The only communication allowed is via service interface calls over the network.



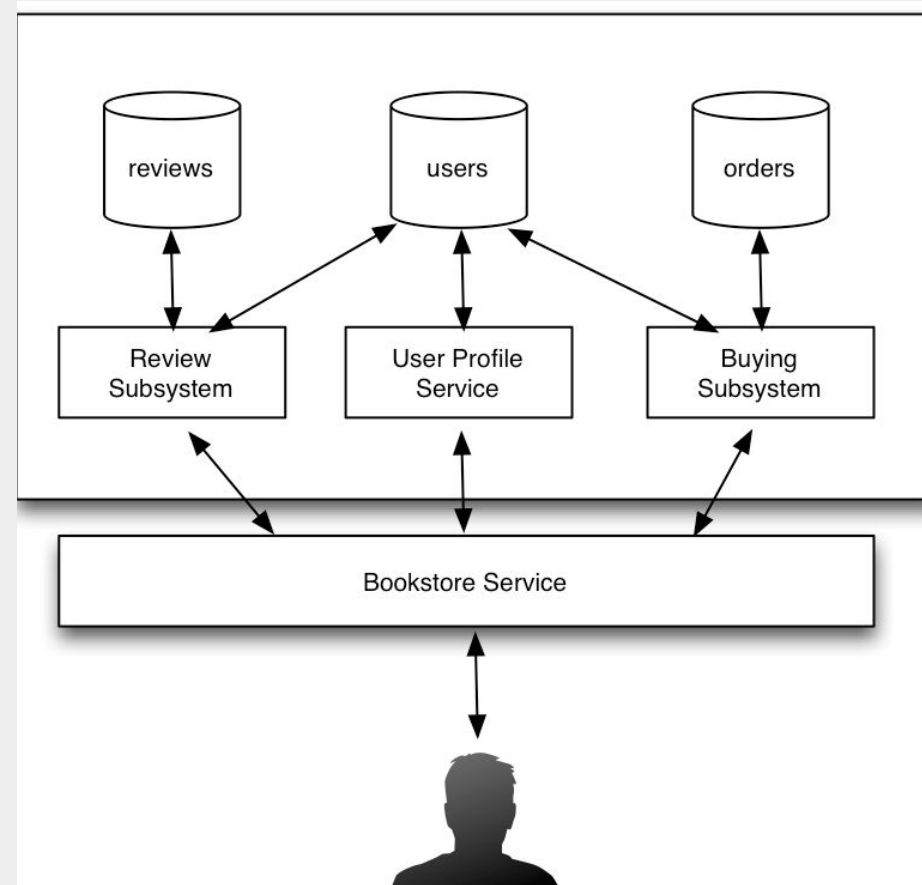
CEO: Amazon shall use SOA!

4. “It doesn't matter what [API protocol] technology you use.
5. “Service interfaces, without exception, must be designed from the ground up to be externalizable. That is to say, the team must plan and design to be able to expose the interface to developers in the outside world. No exceptions.
6. “Anyone who doesn't do this will be fired.
7. “Thank you; have a nice day!”



Amazon: início de SOA!

- Subsistemas internos podem compartilhar dados diretamente
- User profile Service
- Todos os subsistemas “dentro” de uma única API (“Bookstore”)

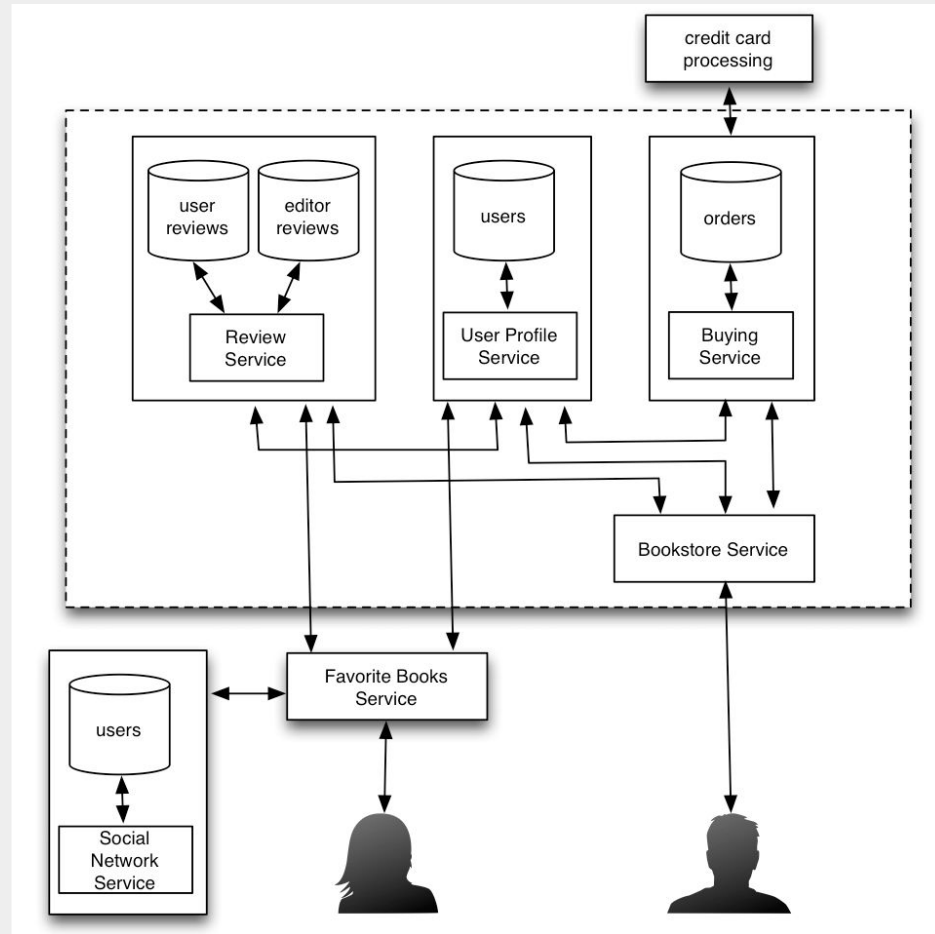


(Figure 1.3, Engineering Long Lasting Software by Armando Fox and David Patterson, Beta edition, 2012.)



Amazon: início de SOA!

- Subsistemas independentes, como se estivessem em datacenters separados
- User Service API
- Pode recombinar serviços para criar novos serviços (“Favorite Books”)



(Figure 1.4, *Engineering Long Lasting Software* by Armando Fox and David Patterson, Beta edition, 2012.)



Pergunta

- Quando dois serviços são compostos em uma SOA, quais condições abaixo **são** necessárias, se houver?
 - A. Ambos devem ser escritos na mesma linguagem/estrutura
 - B. O código que “une” as informações dos dois serviços deve ser código JavaScript em execução no navegador
 - C. Os dois serviços não devem exigir uma etapa de autenticação (login)
 - D. Nenhuma das opções acima são condições necessárias



Pergunta

- Quando dois serviços são compostos em uma SOA, quais condições abaixo **são** necessárias, se houver?
 - A. Ambos devem ser escritos na mesma linguagem/estrutura
 - B. O código que “une” as informações dos dois serviços deve ser código JavaScript em execução no navegador
 - C. Os dois serviços não devem exigir uma etapa de autenticação (login)
 -  D. Nenhuma das opções acima são condições necessárias



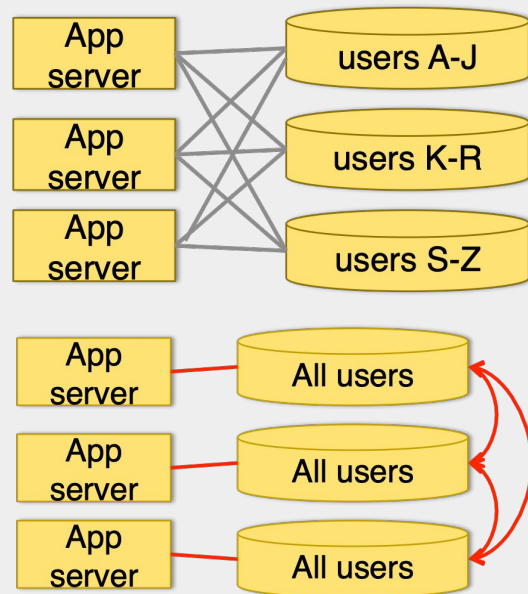
Pós e Contrás de SOA

- Reusabilidade
- Melhor ferramenta para o trabalho
- Mais fácil para testas
- Mais ágil e amigável
- Você constrói, você roda
- Estado dividido entre serviços
- Todos os dados serializados na fronteira
- Mais difícil de testar
- Gerenciamento de desempenho e falhas
- Dev/ops vs. trabalhos separados

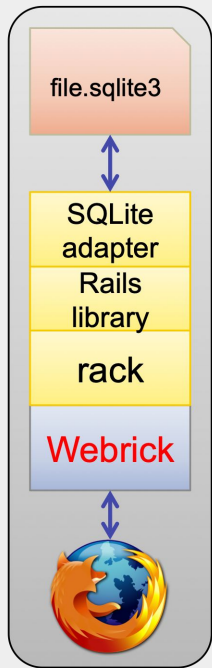


Sharding vs. Replication

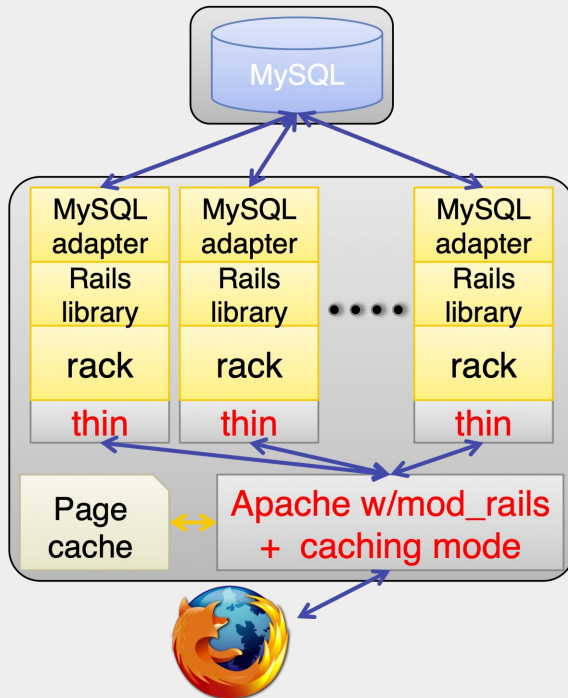
- Particionamento dos dados através dos “shards”
 - + Escala
 - Ruim quando operações acessam >1 tabelas
 - Ex: user profile
- Replicação em todos os lugares
 - + Consultas multi-table rápidas
 - Difícil de escalar: escritas devem ser propagadas (inconsistência temporária)
 - Ex.: Posts no mural facebook / “likes”



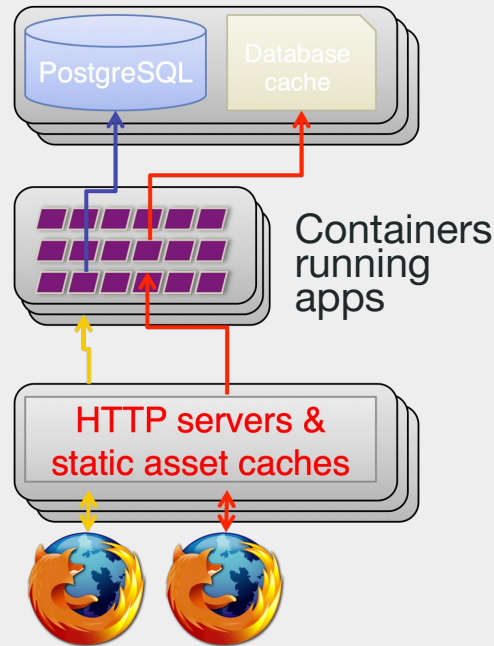
Ambiente de Desenvolvimento vs. Implantação em Média-Escala



Developer



Medium-scale deployment



Large-scale curated deployment, e.g. Heroku



Vocabulário do arco: mesma palavra, significados diferentes

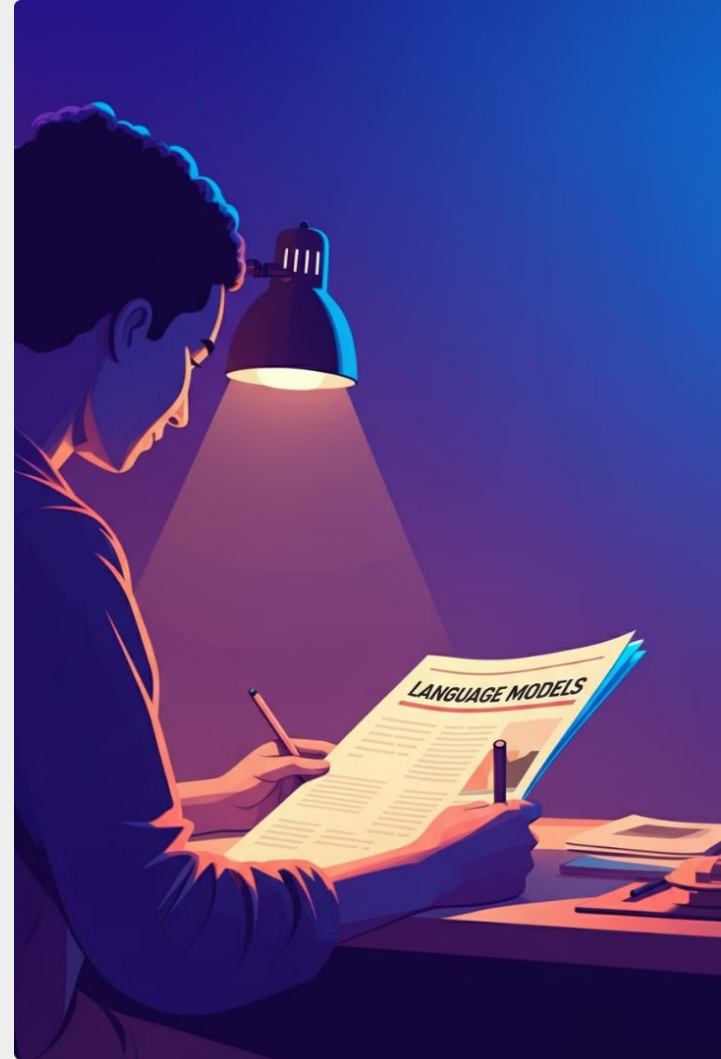
Termo	Hoje	Onde o significado muda
Service	Componente que expõe uma funcionalidade por uma interface de rede (SOA)	Princípios de design: camada de lógica de negócio no código (services/). Estilos arquiteturais: microsserviço, unidade de implantação independente
Component	Parte substituível de um sistema, com responsabilidade definida	C4: componente é um módulo dentro de um contêiner, não um componente de UI (React)
Contêiner	Ambiente isolado que empacota e executa uma aplicação (slide anterior)	C4: contêiner é um processo em execução independente (API, frontend, banco), não necessariamente Docker
Architecture	Organização de alto nível: camadas, serviços, protocolos	Estilos arquiteturais e seus trade-offs; depois, a arquitetura documentada em diagramas (C4)

Quando alguém disser “service” ou “contêiner”, pergunte: em que nível de abstração?



Próximos passos

- Eng. Software em Dimensões
 - Cap. 14 (seções 14.1–14.8):
Arquitetura, C4 Model,
qualidade arquitetural
- Eng. Software Moderna
 - "Cap. 5 — Princípios de
Projeto Coesão, acoplamento,
SOLID
 - Cap. 7 — Arquitetura Camadas,
MVC, trade-offs"



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
[viniciusgarcia.me](https://www.viniciusgarcia.me)

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Introdução ao Design e Projeto de
Arquitetura de Software